Date:
Page:

Module - 3

Sol No 1 Logic :-

→ Divide the array into two halves until each subarray has one element

→ Merge the sorted subarray ~~sub~~ by comparing their elements

→ Repeat the process until the complete array is sorted.

Approach :-

→ Read the number of package values

→ Divide the array recursively into smaller subarrays

→ Merge the sorted subarrays in ascending order

→ Print the sorted package value.

Dry Run :-

N=6

450 1200 800 650 300 1500

Step	array
Initial	450 1200 800 650 300 1500
Divide	[450 1200 800] [650 300 1500]
Merge left	450 800 1200
Merge right	300 650 1500
Final merge	300 450 650 800 1200 1500

Date:
Page:

Soln-2 Logic

→ Divide the array into smaller subarrays using merge sort

→ Merge the subarrays in ascending order

→ Find the median from sorted array

→ Count the values greater than the median and calculate the difference btw the maximum and minimum values.

Approach

→ Read the number of orders

→ Store all processing times in an array

→ Sort the array using merge sort

→ Find the median, count values above it, and calculate the difference

→ Print all required results.

Dry Run n=7 12 75 18 30 15 20 40

Sorted array:-

12 15 18 22 25 30 40

Median = 22

orders above median = 25, 30, 40 → 3

Difference = 40 - 12 = 28.

Soln-5

Date:
Page:

Soln-3 Logic

→ Select the last element as the pivot
→ Place the pivot at its correct position by moving smaller elements to the left and larger elements to the right
→ Recursively apply the same process to the left and right sub arrays

Approach
→ Read the number of response time
→ Store the respons times in an array
→ Partition the array around a pivot
→ Recursively sort the left and right subarrays

Dry run N = 6 120 45 78 200 60 95

step	Array
initial	120 45 78 200 60 95
Pivot	45 78 60 95 120 200
Sort left	45 60 78
Sort right	120 200
Final	45 60 78 95 120 200.

Date:
Page:

Soln-4 Logic

→ Choose the last element as the pivot
→ Place lower values before the pivot to store in descending order
→ Recursively sort the left and right subarrays
→ Find the top 5 values, their avg, and count values above the overall percentage

Approach

→ Read the trade value
→ Sort them in descending order using quick sort.
→ Print the sorted array and top 5 values

Dry Run

N = 8

500 1200 700 2000 900
1500 3000 2500

Sorted :- 3000 2500 2000 1500 1200 900 700 500

Top 5 :- 2000 2500 2000 1500 1200
Top 5 Avg :- 2040
overall Avg :- 1537.5
values above overall avg :- 4.

Date:
Page:

Soln-5 logic :-

→ Build a max heap from the given player scores
→ Swap the root with the last element
& reduce the heap size and restore the heap property using heapify ().
→ Repeat until all scores are sorted in ascending order.

Approach :-

→ Read the number of player scores (N)
→ Store the scores in an array
→ Build a max heap using heapify ()
→ Swap the root with the last element and heapify the remaining heap.

Dry Run :- N=6 850 1700 950 600 1500 1000

Step	array					
Initial	850	1700	950	600	1500	1000
build max heap	1500	1700	1000	600	850	950
Pass 1	1700	950	1000	600	850	1500
Pass 2	1000	950	850	600	1700	1500
Pass 3	950	600	850	1000	1700	1500
Pass 4	850	600	950	1000	1700	1500
pass 5	600	850	950	1000	1700	1500

Date:
Page:

Soln-b Logic :-

→ Build a max heap from the response time
→ Repeatedly swap the root with the last element and heapify the remaining array.
→ After sorting, find the fastest, slowest, avg response time count the ones below avg and cal their percentage.

Approach

→ Read the number of response time
→ Store the response time in an array
→ Sort the array using map sort
→ calculat the fastest, slowest, avg, count and percentage

Orig arr N=8 12 7 15 9 20 5 10 8

Sorted array :- 5 7 8 9 10 12 15 20

operation	Result
Fastest	5
Slowest	20
average	10.75
Ones faster than avg	5 (5,7,8,9,10)
Percentage	(5/8) * 100 = 62.50%

Soln-3 Logic

→ Select
→ Place
Small
to
→ Recur
right

Approa
→ Read
→ Store
→ nu
→ Re

Orig

Sto
init
Pivot
Sort
Sort
Fi